

Governance Control for Routed LLM Applications on Kubernetes: A Measured AKS Systems Study

Abstract

This article reports a measured AKS systems study of a Kubernetes-native governance control plane for routed LLM applications. The implementation combines a controller, Envoy Gateway, Envoy AI Gateway, Azure OpenAI and Azure AI Foundry backends, policy custom resources, and workload-level telemetry. On the current cluster, the system produces live FinOps attribution, executes quality-gate jobs with explicit accept, reject, and abstain outcomes, tracks workload-scoped budget state, emits declared-residency findings, actuates human-approved route changes, and detects labeled gateway-bypass traffic from Tetragon-backed egress evidence. The strongest supported claim is therefore functional: a Kubernetes control plane can reconcile several governance signals and persist auditable evidence on a live cluster. Two follow-up campaigns refine the limits of that claim. An E2 routing-baseline campaign exposed that comparative routing experiments on managed endpoints remain unusable unless quota regime is controlled explicitly, because the present run saw 441 upstream HTTP 429 responses across 512 requests. An E8 scale campaign provided only small-profile descriptive latency numbers and then hit a single-node scheduling limit before controller-scale behavior could be characterized. The present study does not establish routing superiority, inferential quality guarantees, shadow-AI precision/recall, or production-scale performance.

1 Introduction

Networked LLM applications are increasingly deployed as Kubernetes workloads rather than isolated API clients. In that setting, the practical question is not only which model to call, but how to attach governance to each call at the workload level: cost attribution, declared-residency policy, budget state, route-change safety, and detection of traffic that bypasses the approved gateway. This article studies a control-plane approach in which those concerns are represented as Kubernetes custom resources and reconciled against live gateway and egress telemetry [19, 17, 18, 40, 39].

The paper focuses on a systems question rather than on model-level benchmarking: when governance is expressed as Kubernetes resources and enforced through a live gateway stack, which governance signals can actually be observed, persisted, and acted upon on a real cluster, and which parts of the loop fail first under live provider and infrastructure constraints? On AKS we deployed the operator, the Envoy data plane, multiple demo applications, Azure OpenAI and Foundry-backed models, a lightweight Prometheus instance, and Tetragon [32, 23, 31, 33, 30, 44, 46]. That live deployment supports five measured capabilities:

- workload-level FinOps attribution from real gateway traffic,
- quality-gate execution with persisted evidence,
- budget tracking and phase progression,
- declared-residency reporting for non-compliant flows, and
- shadow-AI detection for direct LLM egress outside the gateway.

The main empirical contribution of the current version is therefore functional rather than comparative: live attribution, gate outcomes, budget states, route changes, and rogue-egress findings are all materialized as cluster objects or logs on AKS. The follow-up E2 and E8 campaigns are retained as bounded methodological evidence about the current evaluation environment, not as completed comparative or scalability results.

Several stronger claims remain out of scope. This paper does not establish routing superiority over external baselines, inferential quality guarantees with justified sample sizes, end-to-end approval security, shadow-AI precision/recall, or production-scale performance. That narrower framing is consistent with FinOps governance guidance and with emerging AI risk-management expectations that emphasize traceability and policy controls in addition to raw model quality [9, 7, 8, 34, 41].

2 Related Work

Existing systems in this problem space usually optimize one layer at a time: model routing, policy enforcement, gateway brokering, or cost attribution. Cost-quality routers such as RouteLLM and FrugalGPT focus on model-selection efficiency rather than Kubernetes-native governance workflows [35, 1]. More recent routing work continues this direction with difficulty-aware, test-time, and universal routers such as Hybrid LLM, BEST-Route, EcoRouter, and Universal Model Routing [3, 11, 47, 14]. These systems are the right conceptual baselines for routing quality/cost trade-offs, but they do not by themselves expose workload-scoped Kubernetes objects, approval workflows, or gateway-bypass evidence.

System family		Runtime route selection	Kubernetes workload attribution	Auditable control objects	Gateway- bypass visibility
RouteLLM	/	yes	no	no	no
FrugalGPT					
AI gateways		partial	limited	limited	no
Gatekeeper	/	no	partial	yes	no
Kyverno					
OpenCost		no	yes	limited	no
This work		yes	yes	yes	partial

Table 1: Positioning of the current system against adjacent families. “Partial” means the capability exists in a narrower form than the governance loop studied here.

Gateway products such as Envoy AI Gateway, Kong AI Gateway, and Portkey concentrate on traffic brokering, fallback, observability, and provider-facing controls [4, 15, 42]. Kubernetes-native policy engines such as Gatekeeper and Kyverno are powerful for admission control and policy reporting, but they are not runtime LLM route-governance loops [36, 20, 21]. OpenCost provides vendor-neutral Kubernetes cost allocation, which is adjacent to but distinct from runtime governance-aware LLM routing [37, 38]. At the platform layer, the operator pattern, custom resources, and Gateway API provide the control-plane primitives used by this article’s design [19, 17, 16, 18].

The current paper therefore claims a narrower contribution than a full routing paper: a live Kubernetes control plane can reconcile multiple governance signals and persist workload-scoped evidence across routing, budget, quality, and egress paths. The paper is also distinct from evaluation-only frameworks such as HELM, MMLU, GSM8K, and MT-Bench, which inform quality assessment but are not themselves governance control loops [22, 12, 2, 48].

3 System Model and Problem Statement

We model each routed LLM interaction as a workload-attributed request emitted by a Kubernetes application. A request carries at least four governance-relevant attributes: namespace, application, provider/model target, and policy context. The control plane maintains this context through declarative resources:

- **AIProvider** and **AIModel** describe providers, models, zones, and cost priors.
- **AIFinOpsReport** aggregates observed usage into per-workload cost and token summaries.
- **AIQualityGate** evaluates source and candidate models against a golden dataset and records a gate verdict.

- `AIBudgetPolicy` tracks spend against a workload-scoped budget and surfaces a phase such as `WithinBudget` or `Warning`.
- `AISovereigntyPolicy` checks observed flows against declared zone rules and sensitive-data constraints.

The main problem addressed in the current draft is operational, not legal: can this control plane attach workload-scoped governance evidence to live AKS traffic, and can it do so for both approved gateway traffic and rogue traffic that bypasses the gateway? The adversary model considered in the current measurements is a non-malicious or locally misconfigured tenant workload, including a labeled rogue workload that calls an LLM endpoint directly. A malicious cluster administrator remains out of scope. The present study also treats the governance control plane itself as a potential weak point. In the current implementation, `AIChangeRequest` approval is represented by a writable field change, and shadow-egress evidence is relayed through a `ConfigMap`. Those choices are sufficient for functional measurement, but they do not provide cryptographic integrity or separation of duties. Consequently, the paper evaluates whether the loop operates, not whether the control artifacts are tamper-evident against a privileged adversary.

4 Design

The design centers on a closed-loop pattern, although the current measurements cover only part of that loop. Live application traffic is sent through Envoy AI Gateway with workload headers propagated by a lightweight sidecar proxy. The operator consumes the resulting OpenTelemetry-style Prometheus metrics and maps each request to the responsible namespace and application [4, 18, 44, 43, 40, 39]. Separate controllers then materialize governance views:

- FinOps reconciliation aggregates tokens, request counts, model distribution, and cost estimates.
- Quality-gate reconciliation launches evaluation jobs and compares source and candidate observations.
- Budget reconciliation converts observed spend into phases relative to a per-application threshold.
- Sovereignty reconciliation inspects routed flows and, when available, shadow-egress observations derived from Tetragon.

The shadow-AI path is intentionally independent from the gateway. Tetragon observes kernel-level egress events, a forwarder converts those events into the `shadow-egress` `ConfigMap`, and the operator classifies direct calls to known LLM endpoints [46, 45]. This separation matters because a gateway-only system cannot observe traffic that never reaches the gateway.

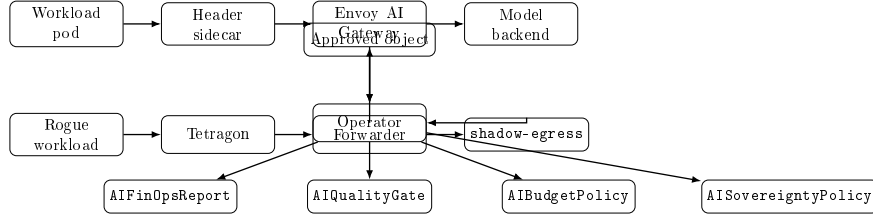


Figure 1: Architecture of the measured governance loop. The current paper evaluates all four paths, but with different depths of evidence.

5 Measured Quality and AIQualityGate

The implemented **AIQualityGate** is an auditable safety gate for route changes. It is not a learned black-box router and it is not a pure LLM judge. Instead, it combines measured evidence and operational telemetry into a composite score with explicit insufficiency behavior. This positioning is closer to governance-oriented evaluation than to benchmark-only reporting, even though benchmark suites such as HELM, MMLU, GSM8K, and MT-Bench motivate some of the signal choices [22, 12, 2, 48].

The default AI Quality Score is:

$$Q = 0.40 \cdot \text{Correctness} + 0.20 \cdot \text{Reliability} + 0.15 \cdot \text{Latency} + 0.15 \cdot \text{Semantic} + 0.10 \cdot \text{Judged}$$

with negative weights clamped to zero and the remaining weights normalized. When judge mode is disabled, the judged weight is removed before normalization. In this paper, however, the composite score is treated as an operator policy heuristic rather than as a validated scientific metric. The weights are implementation defaults, not values justified by sensitivity analysis or human-judgment calibration.

Correctness. The correctness dimension is computed from golden evidence. The implemented checks can combine normalized exact match, reference-vs-actual similarity, required-keyword presence, JSON validity, and field-level F1. This is an auditable deterministic evidence path, not a hidden online model score.

Reliability. Reliability is derived from live telemetry as:

$$100 \cdot \left(1 - \frac{\text{errors} + \text{timeouts} + \text{invalidJSON}}{\text{requests}} \right)$$

when request telemetry is available.

Latency. Latency uses a threshold-based score: latency at half the threshold maps to 100, latency at the threshold maps to 50, and latency at twice the threshold maps to 0, with linear interpolation between those points.

Semantic and judged signals. Semantic and judged scores are not fetched magically by the scoring engine. They are injected through the collected evidence. Judged score is therefore an auxiliary acceptability signal when enabled, not proof of objective correctness.

Insufficient-data behavior. If a weighted signal is missing, or if source/candidate evidence falls below `minSamples`, the gate returns `insufficient-data`. This is a key design property: the operator is designed not to approve a downgrade when the required evidence has not been observed.

Base decision rule. Let Q_s be the source-model score, Q_c the candidate-model score, and τ the configured `tolerancePoints`. The base decision is:

$$\text{candidate-safe} \iff Q_c \geq Q_s - \tau$$

Otherwise the verdict is `candidate-risk`. In the current code path, defaults are `minSamples=1` and `tolerancePoints=3` when not configured explicitly.

Statistical gate status. The controller code also contains an optional confidence-bound statistical path, but the present paper does not evaluate or rely on that path. The available AKS sample sizes are too small to support a statistically defensible inferential claim, so all result statements in this manuscript should be read as descriptive operational evidence rather than inferential quality proof.

Task-specific metrics. The current live AKS evidence exercises enterprise-style golden prompts plus two objective adapters in the external E2 campaign: GSM8K numeric exact match and MMLU multiple-choice exact match. Those objective signals are reported separately in the results because the composite score itself has not yet been validated as a general-purpose quality metric. For this reason, the manuscript uses the gate to study accept/reject/abstain behavior and not to claim task-general objective correctness from one cluster run alone [12, 2].

The currently mounted golden datasets now contain four application-specific `ConfigMap` datasets (`finance`, `legal`, `marketing`, and `rh`) with 20 curated prompts each in the live E3 run. The repeated AKS campaign enforces `minSamples=20` and measures five repetitions per gate, which is enough to characterize stability versus instability for the tested applications, but still not enough to claim broad task-general calibration beyond this portfolio.

qualityeval evidence flow. The `qualityeval` job calls the production gateway endpoint for both the source and candidate models, serializes response evidence, and stores the material consumed by the gate. In Article 2, quality claims are therefore tied to real gateway-backed evaluation rather than offline fabricated comparisons.

Integration with routing and budget enforcement. The intended control loop is that routing or budget-driven candidate changes are safe only when quality evidence is adequate. The current implementation does not yet enforce that invariant on every route mutation path, which is particularly important for declared-residency enforcement. The results section therefore separates what the gate measured from what the controller actually enforced.

6 Implementation

The current implementation was exercised on a real AKS cluster rebuilt on 2026-07-08. The live stack follows the Kubernetes operator pattern on AKS and uses Azure identity and observability building blocks that are standard in Microsoft guidance [19, 17, 32, 23, 29, 27, 26]. The live stack includes:

- the operator image `ghcr.io/ihsenalaya/ai-sovereign-finops-operator/controller:0.5.12`,
- Envoy Gateway and Envoy AI Gateway,
- Azure OpenAI France and US deployments,
- an Azure AI Foundry endpoint used for the EU-backed Mistral path,
- an initial four-application measured subset (`rh/chatbot-rh`, `finance/risk-assistant`, `legal/contract-review`, and `marketing/content-writer`),
- a later portfolio extension to 13 normal gateway-routed applications and 2 rogue applications, with labeled deployments recorded on AKS,
- a lightweight in-cluster Prometheus deployment for operator scraping, and
- Tetragon with an egress tracing policy plus a labeled rogue workload.

Reproducible run folders were generated for FinOps attribution, quality gates, budget tracking, declared-residency reporting, workflow state capture, and shadow-AI detection. The model-serving side combines Azure OpenAI access patterns with Azure AI Foundry-hosted endpoints and observability surfaces [31, 33, 24, 30, 25, 28].

7 Methodology

This paper reports measured AKS behavior from one rebuilt cluster on 2026-07-08. The reported runs are artifact-driven and exploratory rather than statistically complete, and all headline claims are restricted accordingly.

The live methodology used here is straightforward:

- deploy the operator, gateway, providers, and applications on AKS;
- generate real application traffic through the gateway from workload pods;

- collect operator reports, gateway metrics, quality-gate ConfigMaps, budget policy status, and declared-residency findings;
- deploy Tetragon and a labeled rogue workload that calls `api.openai.com` directly;
- convert observed egress events into `shadow-egress` and verify that the operator emits a shadow-AI event.

Instrumentation is grounded in standard telemetry stacks rather than bespoke tracing logic: Prometheus captures gateway-facing metrics, OpenTelemetry conventions inform signal naming, and Tetragon provides kernel-level egress evidence outside the gateway path [44, 43, 40, 39, 46]. Cost interpretation is aligned with OpenCost-style cluster attribution concepts and FinOps governance terminology, while the AI provider layer is supplied by Azure OpenAI and Azure AI Foundry [37, 38, 7, 6, 24, 30].

The rebuilt AKS environment now includes 13 normal gateway-routed applications across multiple namespaces and 2 rogue applications. Only a subset has deep measured evidence, and the manuscript distinguishes clearly between deployed coverage and experimentally exercised coverage.

Two additional campaigns were executed specifically to reduce the earlier gaps. E2 issued 512 live gateway requests across eight routing baselines, with 60 objective items and 4 open-ended items per baseline. E8 deployed workload profiles of 12, 50, and 100 applications on the same AKS cluster and measured gateway latency from the application side. The current run set is sufficient for descriptive systems reporting, but not for inferential statistics, multi-cluster generalization, or robust false-positive/false-negative estimation. This is especially important because routing baselines in the literature are usually evaluated on benchmark suites or offline traces rather than on a live governed gateway stack [35, 1, 3, 11, 47, 14].

The artifact surface is explicit. The paper-level evidence is backed by versioned manifests, run folders, and replay scripts under `article2/experiments/`, `article2/scripts/`, and `article2/reports/`. In particular, E2 and E8 are backed by `run_e2_routing_campaign.py` and `run_e8_performance_campaign.sh`, while quality-gate and control-path evidence is captured in the dated AKS run folders and the claims-to-evidence indexes.

8 Results

The current AKS deployment demonstrates that the governance control plane is operational on live Azure infrastructure. The strongest observations from the latest run are descriptive and directly traceable to Kubernetes resources and captured logs.

FinOps attribution. The live `AIFinOpsReport` for `ai-report-all` reached a total estimated cost of 0.022599 EUR with 1,842 input tokens and 13,083

ID	Strongest current result	Status
E1	Live <code>AIFinOpsReport</code> with per-application cost, request, token, and recommendation fields on AKS	Partial
E3	Repeated five-run quality-gate campaign with 20-prompt golden datasets, quantified stability, and two <code>insufficient-data</code> controls	Partial
E4	Live budget phases observed through <code>Warning</code> , <code>Critical</code> , and <code>Exceeded</code>	Partial
E5	Report-only findings plus enforce-mode FR-only reroute and rollback	Partial
E6	Live <code>AIRouteOverride</code> , approved actuation, and rejected no-op <code>AIChangeRequest</code> behavior	Partial
E7	Tetragon-backed gateway-bypass detection with two true-positive operator warning paths	Partial
E2	Eight-baseline live routing campaign completed, but comparative conclusions are confounded by upstream 429 throttling	Partial
E8	12-app and 50-app profiles completed; 100-app profile blocked by single-node pod-density limits	Partial

Table 2: Summary of the current Article 2 experiment status on AKS.

output tokens. The dominant cost contributor was `finance/risk-assistant` on `gpt-us-mini`, with 660 requests and 0.016477 EUR attributed to a non-compliant US provider. The report also retained per-application routing-score entries and application-specific cost-saving recommendations.

A later post-extension portfolio snapshot already shows that attribution is not limited to the original four-app subset. In that later report, new portfolio applications such as `finance/invoice-extractor` and `legal/compliance-classifier` appeared as separate application entries with their own request counts, costs, and routing-score rows. This is useful as an implementation milestone because it demonstrates that the gateway attribution path scales beyond the earliest smoke-test tenants, even though the full prompt-level routing baseline study remains incomplete.

Quality gates. The updated 2026-07-08 AKS quality campaign replaced the earlier one-prompt run with four application-specific golden datasets of 20 curated prompts each, `minSamples=20`, and five repetitions per gate. The resulting evidence is no longer a single-shot controller demonstration; it quantifies stability under live response variance. Two gates were stable-safe across all five repetitions: `finance-risk-assistant-quality` averaged 73.219 with standard deviation 1.017, and `marketing-content-quality` averaged 73.832 with standard deviation 0.418. One gate was stable-risk across all five repetitions: `rh-chatbot-quality` averaged 69.495 with standard deviation 0.345 and never crossed the tolerance boundary. The most informative case was

Gate	Target	Stability	Verdict	pat-tern	Mean	Std	N	Reps
finance-risk-assistant-quality	finance/risk-assistant	stable-safe	5	safe / 0	73.219	1.017	20	5
legal-contract-quality	legal/contract-review	unstable	4	safe / 1	66.013	1.534	20	5
marketing-content-quality	marketing/content-writer	stable-safe	5	safe / 0	73.832	0.418	20	5
rh-chatbot-quality	rh/chatbot-rh	stable-risk	0	safe / 5	69.495	0.345	20	5
			risk					

Table 3: Detailed AKS AIQualityGate outcomes from the latest E3 campaign.

legal-contract-quality: it produced one **candidate-risk** verdict followed by four **candidate-safe** verdicts, yielding an unstable stability label with mean 66.013 and standard deviation 1.534. We also retained two auxiliary gates: one without **evidenceRef**, which remained **insufficient-data**, and one with a non-compliant candidate lacking both evidence and telemetry, which also remained **insufficient-data**. The robust conclusion is therefore stronger than simple accept/reject coverage: the live AKS run now demonstrates reproducible safe gates, reproducible risky gates, quantitatively unstable gates, and explicit abstention paths on real evidence.

Application coverage. The deployed portfolio now contains 13 normal applications and two rogue workloads, but only a measured subset has been exercised deeply enough for the strongest result statements. The generated application matrix makes that boundary explicit: **finance/risk-assistant**, **legal/contract-review**, and **marketing/content-writer** are already tied to E1/E2-ready workload definitions, while most extension applications are currently only E3-ready from a policy and dataset perspective. This distinction matters because it prevents us from overstating multi-tenant evaluation breadth.

Budget tracking. The most informative budget result is **finance-budget**. Under real traffic it entered the **Warning** phase at 86% of a 0.02 EUR budget, with 0.017104 EUR current spend. We then captured a controlled phase progression on the live cluster by tightening the policy thresholds: with **criticalThresholdPercent=80** and **hardLimitPercent=95**, the same observed spend moved the policy to **Critical**; after temporarily lowering **budgetEUR** to 0.017, the policy moved to **Exceeded** at 101%. The operator emitted matching **BudgetThreshold** warning events for both transitions. The other three application budgets remained within budget. This supports live phase-transition behavior, but not automatic fallback actuation, because no cheaper observed managed fallback model exists in the current catalog.

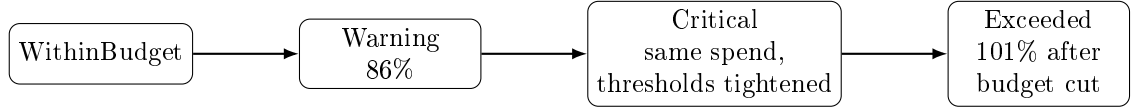


Figure 2: Observed budget-policy phase progression for **finance-budget**. The latter two transitions were induced by threshold and budget changes, not by additional spend.

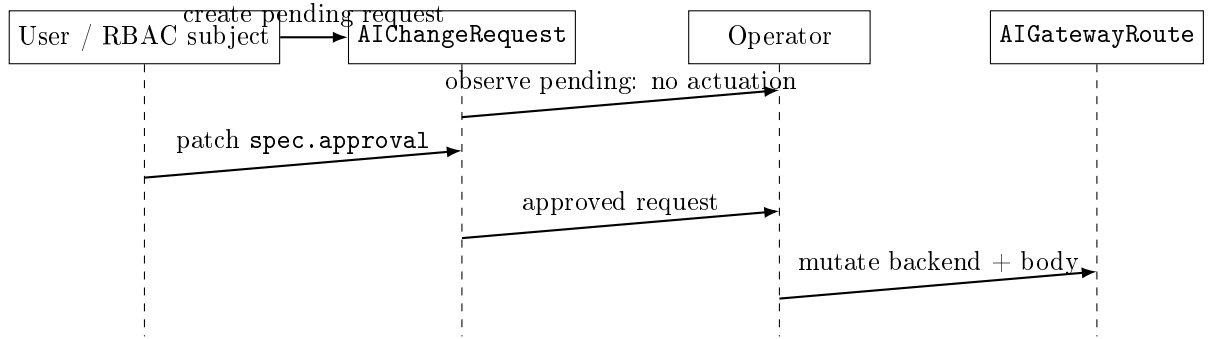


Figure 3: Measured **AIChangeRequest** path. The current implementation gates actuation on a writable approval field; it does not provide signature-backed approval integrity.

Human workflow and reroute actuation. After publishing operator image `ghcr.io/ihsenalaya/ai-sovereign-finops-operator/controller:0.5.12`, a live `AIRouteOverride` actuated a reroute from `gpt-us-mini` to `gpt-france-mini`. The corresponding `AIGatewayRoute` changed backend from `greenops-openai-us` to `greenops-openai-fr` and added a `bodyMutation` that rewrote the request model field. Deleting the override restored the original route. We then exercised `AIChangeRequest`: in `Pending`, the route remained unchanged; after patching `spec.approval=Approved`, the request became `Actuated` and applied the same reroute. In a separate rejected-path run, the request stayed non-actuating while `Pending` and the route still remained unchanged after `spec.approval=Rejected`. This demonstrates live human approval gating, explicit rejection behavior, and route actuation, but not automatic rollback on deletion, which is not implemented in the current controller.

Declared-residency reporting. The current `AISovereigntyPolicy` produced both report-only and enforce-mode evidence. In `reportOnly`, the operator emitted a critical finding and a warning for `finance/risk-assistant` traffic sent to `openai-us` in the forbidden US zone. We then enabled `enforce`. With `allowedZones={FR,EU}`, the controller first selected the cheapest compliant target (`mistral-small`) but did not mutate the route because that target was not routable in the live gateway stack. After temporarily narrowing `allowedZones`

to FR, the cheapest compliant routable target became `gpt-france-mini`, and the operator actuated a real reroute of `greenops-openai-us` to `greenops-openai-fr` with an `enforced-reroutes` annotation and request-body model rewrite. Restoring `reportOnly` restored the original route automatically. This is a stronger result than report-only detection alone because it both demonstrates enforce-mode capability and exposes an implementation limit that must be discussed honestly.

E2 routing baselines. The live E2 campaign completed after we repaired stale gateway backends and stale provider API-key secrets in the cluster. The resulting run issued 512 requests through the AKS gateway across eight routing baselines, with 60 objective items and 4 open-ended items per baseline. The dominant finding is not a winning baseline but a confounder: 441 of the 512 requests failed with upstream HTTP 429 responses, concentrated on the `gpt-france-mini` deployment in `francecentral`. Under that throttled regime, objective exact-match values ranged from 0.000 to 0.133 and error rates ranged from 0.688 to 0.969 across baselines. The campaign is therefore still informative for systems analysis because it shows that the comparative routing experiment was blocked by provider quota behavior rather than by missing control-plane automation. It does not support superiority, inferential ranking, or even stable pairwise ranking claims among baselines.

E8 performance and scale. The E8 campaign measured gateway-facing application latency under increasing numbers of deployed workloads. The 12-application profile completed 12 requests with no errors and p50/p95/p99 latencies of 165.0/1468.1/1558.9 ms. The 50-application profile completed 50 requests with four errors and p50/p95/p99 latencies of 255.5/25193.9/26696.9 ms. The 100-application profile did not complete because the single-node AKS cluster hit a scheduler pod-density limit: 74 pods remained pending, 26 were running, and the scheduler repeatedly reported `0/1 nodes are available: 1 Too many pods`. These latency percentiles are descriptive only: with 12 and 50 requests, the upper-tail numbers are too small-sample to support a scalability claim, and the 50-application tail is plausibly contaminated by the same provider throttling seen in E2. The current artifact therefore documents an incomplete scale attempt, not a completed overhead study of the controller itself.

Shadow-AI detection. After deploying Tetragon and a labeled rogue workload, the `shadow-egress` ConfigMap recorded `finance/rogue-script -> api.openai.com` with eight direct connections. The operator log then emitted a `ShadowAI` warning for direct forbidden-zone egress that bypassed the gateway. A second rogue workload in the `shadow` namespace generated direct `api.anthropic.com` egress. Tetragon export logs captured that traffic as repeated `tcp_connect` observations, and after reconstructing the bridge input using the same forwarder logic, the operator emitted repeated `ShadowAI` warnings for `shadow/rogue-direct-llm-2 -> api.anthropic.com`. Across those captured bridge snapshots, no normal

gateway-routed application appeared in the `shadow-egress` payload. This strengthens the current evidence beyond a single endpoint family, while also revealing that the continuous eBPF-to-ConfigMap bridge must be treated as an explicit operational dependency.

Scope note. The limits of E2, E3, E7, and E8 are consolidated in Section 10 rather than repeated here.

9 Discussion

The current evidence suggests that the most immediately useful aspect of the system is not automatic route optimization but unified visibility. Even without turning on enforcement modes, a platform team can already observe who spent money, which application violated a declared zone policy, which candidates failed a quality gate, and whether an application bypassed the gateway entirely. That emphasis on observability and governed change aligns more closely with FinOps and AI risk-management practice than with pure benchmark chasing [9, 7, 8, 34, 13].

The completed E2 campaign sharpens that point. Once stale backends and stale API-key secrets were repaired, the limiting factor was no longer the controller path but upstream provider throttling. For this paper, that observation is best read as a protocol warning about live managed-endpoint evaluation rather than as a comparative routing result.

The completed E8 campaign reveals a different weakness. On the current single-node cluster, the experiment became pod-density bound before there was evidence of controller saturation. The jump from the 12-application profile to the 50-application profile also showed a long-tail latency increase and a non-zero error rate, but with only one request per application that tail is better interpreted as a warning about protocol contamination than as a performance characterization.

Another useful observation is that quality-gate behavior and budget behavior are naturally coupled, but the current implementation does not enforce that coupling on every actuation path. In the present run, the finance workload accumulated enough spend to reach a warning state before any automated fallback claim could be made. Separately, the residency-enforcement reroute changed the served model without first passing through the same quality-gate path that the design recommends for route changes. That is a useful negative result: the policy loop can act, but the safety invariant “no route change without adequate evidence” is not yet enforced end to end.

The shadow-AI follow-up also exposed a practical systems lesson. The eBPF signal itself was present for both OpenAI and Anthropic rogue traffic, but the evidentiary bridge into the operator depended on the `shadow-egress` handoff remaining populated from current Tetragon exports. In other words, the hard problem was not endpoint recognition but operationalizing the bridge as a continuously refreshed telemetry path.

The manuscript also has to be explicit about control-plane security limits. In the present implementation, `AIChangeRequest` approval is a writable field transition and the `shadow-egress` evidence path terminates in a mutable `ConfigMap`. Those choices are acceptable for a functional systems prototype, but they are not enough for a strong security claim. The right interpretation is therefore that the paper measures governance functionality under a limited adversary model, not tamper-evident governance under privilege compromise.

That limitation also suggests the clearest path to a stronger identity for the paper. The project already points toward signed evidence and stronger approval semantics; integrating signature-backed admission and evidence verification would turn a current caveat into a measurable control-plane property rather than leaving it as future work by implication.

10 Validity and Scope

This study is intentionally narrow. It supports live AKS claims about FinOps attribution, quality-gate execution, budget tracking through `Warning`, `Critical`, and `Exceeded`, declared-residency reporting plus an FR-only enforce-mode reroute, human approval gating with route actuation, true-positive shadow-AI detection on labeled rogue workloads, an eight-baseline routing campaign confounded by provider throttling, and small-to-medium performance profiles on one cluster. The compliance framing is deliberately limited to configured policy enforcement and declared-residency observability rather than formal legal certification, which is important when reading the work against GDPR, the EU AI Act, NIST AI RMF, OWASP guidance, and ISO/IEC 42001 [10, 5, 34, 41, 13]. It does not yet support:

- final routing-tradeoff claims against a full baseline family,
- confidence-interval-backed or other inferential headline numbers,
- broad residency-enforcement claims that ignore live catalog routability limits and quality-gate bypasses,
- tamper-evident approval or audit-integrity claims,
- full shadow-AI precision/recall characterization or adversarial evasion coverage, or
- production-scale performance and scalability claims.

Additional validity threats are specific and concrete: all evidence comes from one cluster rebuilt on one day; the E2 campaign was dominated by Azure OpenAI 429 throttling on the France deployment; the E8 large profile was blocked by a single-node scheduler limit rather than by a multi-node stress test; the improved E3 campaign still covers only four application-specific datasets despite raising them to 20 prompts and five repetitions, and one gate (`legal-contract-quality`)

remained verdict-unstable across repetitions; and the current shadow-AI evidence is true-positive only. Accordingly, the paper should be read as a measured AKS systems study with a deliberately narrowed claim set, not as a final comparative routing, security, or scalability paper.

11 Conclusion

The rebuilt AKS deployment now supports a clearer systems paper than the earlier partial draft. Real workloads, real Azure model endpoints, and real operator reconciliations demonstrate live FinOps attribution, quality-gate execution, budget-state progression, declared-residency findings, route actuation, and shadow-AI detection. The central contribution of the present version is this functional governance loop on a live cluster; the remaining comparative, statistical, and scalability limits are explicitly consolidated in Section 10.

References

- [1] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [3] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid llm: Cost-efficient and quality-aware query routing. In *The Twelfth International Conference on Learning Representations*, 2024.
- [4] Envoy AI Gateway Authors. Envoy ai gateway documentation. Project Documentation, 2026.
- [5] EUR-Lex. Rules for trustworthy artificial intelligence in the EU. Legal Summary, 2026.
- [6] FinOps Foundation. Data ingestion. FinOps Framework Capability, 2026.
- [7] FinOps Foundation. Finops framework. Framework Documentation, 2026.
- [8] FinOps Foundation. Governance, policy & risk. FinOps Framework Capability, 2026.
- [9] FinOps Foundation. What is finops? FinOps Framework, 2026.
- [10] GDPR-Info.eu. General data protection regulation (GDPR). Legal Text Portal, 2026.

- [11] Kaixin Go, Uri Gadot, Navdeep Kumar, Kfir Yehuda Levy, and Shie Mannor. Best-route: Adaptive llm routing with test-time optimal compute. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.
- [12] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [13] International Organization for Standardization. Iso/iec 42001 artificial intelligence — management system. Standard Overview, 2023.
- [14] Wittawat Jitkrittum, Harikrishna Narasimhan, Ankit Singh Rawat, Jeevesh Juneja, Congchao Wang, Zifeng Wang, Alec Go, Chen-Yu Lee, Pradeep Shenoy, Rina Panigrahy, Aditya Krishna Menon, and Sanjiv Kumar. Universal model routing for efficient llm inference. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [15] Kong Inc. Kong ai gateway. Product Documentation, 2026.
- [16] Kubernetes Authors. Custom resource definitions. Kubernetes Documentation, 2025.
- [17] Kubernetes Authors. Custom resources. Kubernetes Documentation, 2025.
- [18] Kubernetes Authors. Gateway api. Kubernetes Documentation, 2025.
- [19] Kubernetes Authors. Operator pattern. Kubernetes Documentation, 2025.
- [20] Kyverno Authors. Kyverno introduction. Project Documentation, 2026.
- [21] Kyverno Authors. Kyverno policy reports. Project Documentation, 2026.
- [22] Percy Liang, Rishi Bommasani, Tony Lee, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023.
- [23] Microsoft. Overview of managed identities in azure kubernetes service (AKS). Microsoft Learn, 2025.
- [24] Microsoft. Azure ai foundry documentation. Microsoft Learn, 2026.
- [25] Microsoft. Azure ai foundry models overview. Microsoft Learn, 2026.
- [26] Microsoft. Azure monitor metrics overview. Microsoft Learn, 2026.
- [27] Microsoft. Kubernetes monitoring overview. Microsoft Learn, 2026.
- [28] Microsoft. Monitor agents using the dashboard. Microsoft Learn, 2026.
- [29] Microsoft. Monitor azure kubernetes service (AKS). Microsoft Learn, 2026.
- [30] Microsoft. Observability in azure ai foundry. Microsoft Learn, 2026.

- [31] Microsoft. Secure access to azure openai from azure kubernetes service (AKS). Microsoft Learn, 2026.
- [32] Microsoft. Use a microsoft entra workload id on azure kubernetes service (AKS). Microsoft Learn, 2026.
- [33] Microsoft. What is azure ai foundry? Microsoft Learn, 2026.
- [34] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). NIST Publication, 2023.
- [35] Isaac Ong, Yilun Yao, Tianyu Zhang, Karthik Narasimhan, and He He. Routellm: Learning to route llms from preference data. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [36] Open Policy Agent Gatekeeper Authors. Gatekeeper documentation. Project Documentation, 2026.
- [37] OpenCost Authors. Opencost overview. Project Documentation, 2026.
- [38] OpenCost Authors. Opencost specification. Project Documentation, 2026.
- [39] OpenTelemetry Authors. Opentelemetry overview. Specification Documentation, 2026.
- [40] OpenTelemetry Authors. What is opentelemetry? Project Documentation, 2026.
- [41] OWASP Foundation. Owasp top 10 for large language model applications 2025. Project Documentation, 2025.
- [42] Portkey. Portkey ai gateway documentation. Product Documentation, 2026.
- [43] Prometheus Authors. Prometheus data model. Project Documentation, 2026.
- [44] Prometheus Authors. Prometheus overview. Project Documentation, 2026.
- [45] Tetragon Authors. Tetragon enforcement. Project Documentation, 2026.
- [46] Tetragon Authors. Tetragon overview. Project Documentation, 2026.
- [47] Li Yang et al. Let the llm stick to its strengths: Learning to route economical llm. In *OpenReview*, 2025.
- [48] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.